Design and Analysis of a Multi-Threaded Transaction Processing System

A Comparative Study of Optimistic Concurrency Control and Conservative Two-Phase Locking

Keshav Bhaskarbhai Mehta
Donald Bren School of Engineering and Computer Science
University of California, Irvine
keshavbm@uci.edu

Sahil Gupta
Donald Bren School of Engineering and Computer Science
University of California, Irvine
sahilg07@uci.edu

ABSTRACT

The project develops a multi-threaded transaction processing layer using RocksDB as the underlying key-value database and examines the effectiveness of two concurrency control protocols, namely, Optimistic Concurrency Control (OCC) and Conservative Two-Phase Locking (2PL). The system processes concurrent transactions using multiple threads and evaluates the performance of the protocols using different levels of contention and concurrency. Two workloads were developed for the experiment: a bank transfer workload and a combined NewOrder/Payment workload, which resembles OLTP systems. The experiments evaluate the system's throughput, retry count, and average response time for different thread counts and contention levels. The results show the effectiveness of the two protocols, where OCC performs well for low contention due to its lock-free nature, and Conservative 2PL performs well for high contention by avoiding repeated aborts. This project emphasizes the importance of the trade-off between optimistic and pessimistic concurrency control protocols.

1. INTRODUCTION

Database systems today must allow many simultaneous transaction runs while also ensuring that they execute correctly and consistently against each other. Transaction processing systems permit multiple simultaneous transactions while accessing the same data. However, the concurrent access to that data may create possible conflicts, such as a read being overwritten by a write or a write overwriting another write.

If not properly handled so as to avoid these conflicts, the end consequence may be an inconsistent database or a violation of transactional isolation. To resolve these types of conflicts created by concurrent transactions, concurrency control protocols can be applied to the database management system. Such concurrency control protocols will allow for all transactions to occur as if they occurred one at a time (i.e., serial execution), even if they occurred simultaneously (i.e., concurrent execution). Some concurrency control protocols include the Optimistic Concurrency Control (OCC) and the Two-phase Locking Protocol (2PL). The OCC executes transactions using an optimistic approach, which means that a transaction executes without obtaining a lock prior to execution, but will wait to commit until validating that no conflicts exist.

The first step taken in this project has been to design a multi-threaded transaction processing layer to run on top of the RocksDB key-value storage engine. We will then compare the performance of Optimistic Concurrency Control (OCC) and Conservative Two Phase Locking (2PL) for various workloads by executing concurrently transactions with several worker threads and measuring performance in terms of throughput, response time, and retry rate.

Our objective is to evaluate the behavior of different concurrency control protocols based on their level of contention and parallelism so that we have a better understanding of how these two types of concurrency controls (optimistic vs pessimistic) work together and what their advantages/disadvantages are for transaction processing systems.

2. BACKGROUND

The systems are meant to execute a number of transactions in a way that ensures the correctness of the database. However, when a number of transactions are being processed concurrently, they may end up accessing a common data item simultaneously. This brings about a problem known as interference. The main aim of concurrency control is to ensure that all transactions are processed in a way that is equivalent to a serial order. This is also known as serializability, which is a main objective in a transaction processing system.

2.1 Transaction Processing

A transaction can be defined as a sequence of database operations that are carried out as one formal unit of work. For example,

transferring money between two accounts, and updating multiple related records due to one order. "A correct transaction processing system will preserve the ACID properties" [1]: atomicity, consistency, isolation and durability. Out of these properties of a transaction processing system, the one that is of most relevance for this project is isolation because the project is focused on concurrency control of transactions, and not the recovery of them. Transactions that execute concurrently must not be able to observe each other's intermediate results, as they will produce incorrectly intermediate results that are visible to the other transaction.

When two (or more) transactions execute simultaneously, conflicts can arise if those transactions are using the same data items. The two most commonly encountered types of conflicts are read/write conflicts (one transaction reading while another writes) and write/write conflicts (two transactions attempting to update one data item). These conflicts can cause concurrency anomalies such as, lost updates, dirty reads, or non-repeatable reads if proper concurrency control is not in place.

To prevent concurrency anomalies and to allow transactions to execute concurrently, concurrency control protocols are used. In this project, two commonly used concurrency control protocols are implemented and analyzed: optimistic concurrency control (OCC) and two-phase locking (2PL).

2.2 Optimistic Concurrency Control (OCC)

Optimistic Concurrency Control (OCC) is based on the fact that conflicts between transactions are not very common. OCC does not prevent conflicts by using locks during transaction execution. Instead, it allows the transaction to proceed without locks and performs correctness checks during the commit operation. "OCC is divided into three phases, namely the read phase, validation phase, and write phase" [2]. The read phase involves the transaction reading the data items and storing them in the read set. The updates made by the transaction are stored in a separate write buffer rather than directly updating the database. The validation phase checks whether the transaction is in conflict with the writes that have been committed during the transaction execution period. If the transaction is not in conflict with the writes, the transaction proceeds to the write phase, where the updates are reflected in the database. If the transaction is in conflict with the writes, the transaction is aborted and needs to be retried.

In the current project, OCC is used in the form of sequential validation, where only one transaction is allowed to perform the validation process at a time. OCC eliminates the need for locks and allows for high levels of concurrency with low levels of contention. However, as the level of contention increases, the validation process will fail more often, resulting in the abortion of the transaction and the need for the transaction to be retried.

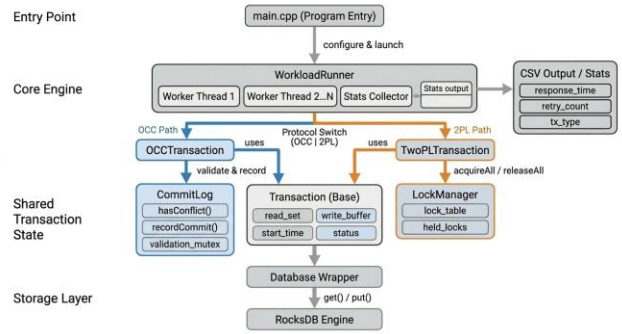
2.3 Two Phase Locking (2PL)

Two-Phase Locking (2PL) takes a pessimistic approach to concurrency, using locks to stop conflicts before they happen. Transactions have to secure a lock before they can touch any data, meaning conflicting tasks are simply put on hold until the lock is free. The process follows two distinct stages: a "growing" phase where the transaction gathers all the locks it needs, and a "shrinking" phase where it lets them go. Once the first lock is released, no new ones can be grabbed. This rigid structure is what guarantees serializable execution.

For this project, I implemented **Conservative Two-Phase Locking** (or static 2PL). In this version, a transaction tries to grab every lock it needs right at the start. If even one is missing, it drops everything and tries again later. Because a transaction never sits around holding some locks while waiting for others, we effectively eliminate the risk of deadlocks [1]. To keep the design clean, the system uses exclusive locks for both reads and writes. While this limits parallel reading, it significantly simplifies the lock manager. Compared to OCC, 2PL trades some speed for stability; it adds the overhead of locking but saves us from the "wasted work" of repeated transaction failures when the system is busy.

3 SYSTEM DESIGN

The current manuscript proposes a multi-threaded transaction processing system, which operates over the RocksDB key-value storage system. This system integrates two different concurrency control protocols, namely, Optimistic Concurrency Control (OCC) and Conservative Two-Phase Locking (2PL), and evaluates the performance of these protocols for different levels of concurrency and contention. This system architecture also modularizes the storage management, transaction processing, concurrency control, and workload generation, so that the storage and workload for both protocols remain the same, enabling a comparative evaluation of the performance of these protocols.



3.1 System Architecture

The system consists of four main layers:

1. Storage Layer
2. Transaction Management
3. Concurrency Control
4. Workload Execution Engine

These components interact to execute transactions concurrently while maintaining database correctness.

3.1.1 Storage Layer

The storage layer is implemented using RocksDB, which provides persistent key–value storage. "A lightweight Database wrapper exposes simple read and write operations that allow higher-level components to access and update records" [3]. All transactional operations ultimately interact with the database through this abstraction.

3.1.2 Transaction Management

The process of transactions can be represented with a fundamental Transaction class that tracks the keys accessed during execution. Each transaction includes:

- a read set, containing keys read during execution
- a write buffer, containing updates to be applied at commit time
- a start timestamp, used for conflict detection

This abstraction allows for the systematic tracking of dependencies required for concurrency control while keeping the execution logic of transactions separate from the protocol used.

3.1.3 Concurrency Control

The system uses two main ways to handle multiple tasks at once: Optimistic Concurrency Control (OCC) and Conservative Two-Phase Locking (2PL). In the OCC approach, the system basically assumes things will go smoothly. Transactions go ahead and do their work without asking for permission or grabbing locks first. As they work, they keep a private "read set" and "write set" to track what data they are looking at and what changes they want to make. These changes aren't saved to the main database right away; they stay in a local buffer. Once the work is done, the transaction enters a validation step where the system checks its notes against a log of other recently completed tasks. If someone else changed the same data while this transaction was running, the validation fails, and the whole thing is tossed out and restarted. However, if everything looks clear, the buffered changes are finally written to the database. This "sequential validation" ensures that only one transaction finishes this final check at a time, keeping everything in a clear, logical order.

On the other hand, Conservative 2PL takes a much more cautious, "better safe than sorry" approach. Before a transaction even starts its work, it has to get a "green light" from a centralized lock manager for every single piece of data it needs. If the manager sees that all the required keys are available, the transaction gets the locks and starts running. If even one key is being used by someone else, the transaction doesn't wait around; it lets go of any locks it already grabbed and tries the whole process again after a short break. Because it insists on having every single lock ready before it moves a finger, it completely avoids deadlocks—those frustrating situations where two tasks are stuck waiting on each other forever. This method uses exclusive locks for both reading and writing, and once the job is finished, the lock manager releases everything so the next task can take its turn. While OCC is faster when there isn't much overlap between tasks, 2PL provides a more disciplined structure that prevents conflicts before they can even start.

3.1.4 Workload Execution Engine

The *WorkloadRunner* coordinates transaction execution. After loading the initial dataset into the database, it constructs a set of keys used by the workloads and optionally defines hot sets of frequently accessed keys. With configurable probability, transactions select keys from the hot set to simulate different contention levels. The system supports two workloads:

- Workload 1 (Transfer) – transactions transfer balance between two accounts.
- Workload 2 (NewOrder + Payment) – transactions update warehouse, district, customer, and stock records.

The runner creates multiple worker threads, each executing a fixed number of transactions according to the selected workload and protocol. This design allows the system to evaluate protocol behavior under different levels of parallelism.

3.2 Statistics Collection

During execution, the system records performance metrics for each transaction, including response time and retry count. These statistics are aggregated once all worker threads complete execution. At the end of execution, the system computes summary metrics such as:

- total number of executed transactions
- total number of retries
- average response time

In addition to printing these metrics, the system writes per-transaction response times to CSV files. These files are later used to generate response-time distribution plots for performance analysis.

3.3 Experiment Workflow

To make the evaluation process more efficient, the team uses an automated script that runs the system over and over again with different settings. Instead of setting things up by hand each time, the script automatically swaps out the protocol being used, the type of workload, how many threads are running, and how much "contention" or competition for data there is. As each test finishes, the system saves all the performance data like how fast it ran and how many errors occurred directly into CSV files.

Once the tests are finished, a separate plotting script takes over to make sense of all those raw numbers. It pulls the data from the CSV files and transforms it into clear visual charts. These plots show the

"big picture," specifically highlighting how throughput (the total work done), retry rates (how often tasks had to restart), and response times (how long each task took) are affected as the system gets busier or more crowded. By using this consistent workflow, it becomes much easier to see the real-world trade-offs between the "go-ahead" style of OCC and the "permission-first" style of Conservative Two-Phase Locking.

4. Evaluation

This section evaluates the performance of the implemented transaction processing system under two concurrency control protocols: Optimistic Concurrency Control (OCC) and Conservative Two-Phase Locking (2PL). The experiments aim to analyze the impact of concurrency and contention on system performance using two representative workloads. The evaluation focuses on three primary metrics:

- Throughput (transactions per second)
- Average transaction response time
- Retry rate

Two workloads are evaluated:

- Workload 1 (Transfer): a simple transfer transaction that updates two account records.
- Workload 2 (NewOrder + Payment): a more complex workload involving multiple reads and updates, representing a higher-contention scenario.

4.1 Experimental Setup

To get the results, all the tests are handled by a single automated script called run_experiments.sh. This script is the "engine" of the evaluation, repeatedly launching the transaction system while swapping out different settings for the protocol and the type of work being done. It performs what's known as a parameter sweep, systematically testing different numbers of worker threads and different levels of data competition. Every time a test finishes, the script captures the performance data and saves it into two main spreadsheets: workload1.csv and workload2.csv.

Each line in those files tells the story of a specific test run. It records which protocol was used (the "go-ahead" OCC or the "lock-first" 2PL), how many threads were working at once, and the "hotset" settings, which determine how often different tasks try to grab the same popular piece of data. To measure success, the script tracks "throughput" (how many tasks finished per second), "average response time" (how long each task took), and the "retry rate" (the percentage of tasks that got interrupted and had to start over).

The experiments are split into two main categories to see how the system handles pressure. First, the team tests "scaling" by adding more and more worker threads moving from 1 up to 16 while keeping the level of data competition steady. This shows if the system actually gets faster as you add more "muscle." Second, they keep the number of threads steady at four but crank up the "hotset probability" from 0.0 to 1.0. This mimics a crowded environment where everyone is fighting over a small handful of popular items, making conflicts much more likely.

Finally, the system goes beyond just looking at averages. It runs special, longer tests to capture the specific timing of every single transaction. These detailed logs are tucked away in a separate folder called results/dist. By looking at this granular data, the team can see if most tasks are finishing quickly or if a few unlucky ones are getting stuck in long delays, which helps paint a much more complete picture of how the system feels in a real-world scenario.

4.2 Plot Generation

After the experiments complete, the script plot_results.py processes the generated CSV files and produces the evaluation figures. The script reads the experimental data using the Python pandas library and generates visualizations using matplotlib.

The following six graphs are generated and used in the analysis:

- Retry Rate vs Contention – Workload 1
- Retry Rate vs Contention – Workload 2
- Throughput vs Threads – Workload 1
- Throughput vs Threads – Workload 2
- Average Response Time vs Threads – Workload 1
- Average Response Time vs Threads – Workload 2

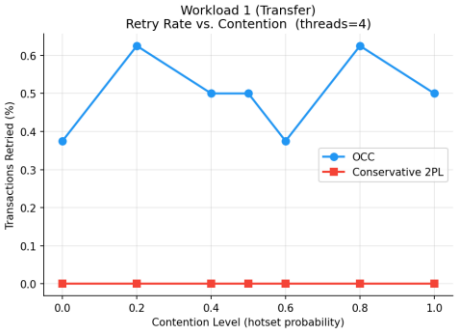
For throughput and latency plots, contention is fixed at hotset_prob = 0.5. For contention experiments, the number of threads is fixed at threads = 4. Each graph aggregates multiple experimental runs by computing the mean performance metric for each parameter value.

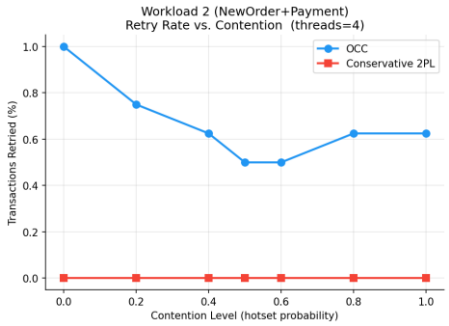
4.3 Experimental Results

4.3.1 Retries vs Contention

The charts comparing Retry Rate vs. Contention clearly show the trade-off between the two strategies. For OCC, the retry rate climbs as the system gets busier. Because transactions "blindly" execute without locks, they only discover they've bumped into someone else at the very end, forcing them to scrap their work and start over. As data competition increases, these mid-air collisions become much more frequent.

In contrast, Conservative 2PL keeps a nearly flat retry rate of zero. By requiring all locks upfront, it essentially organizes transactions into a controlled line. Instead of crashing and restarting, conflicting tasks simply wait their turn to start. Ultimately, the data confirms the classic divide: OCC gambles on a "lock-free" sprint that leads to more wasted effort under pressure, while 2PL uses strict synchronization to ensure every attempt is a successful one.





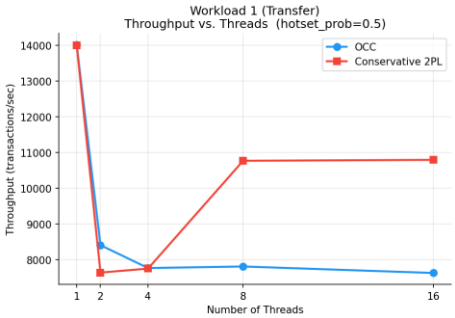
4.3.2 Throughput vs Threads

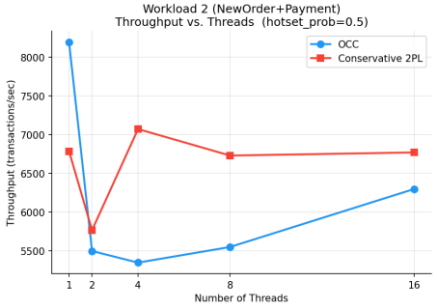
Throughput scalability is evaluated by increasing the number of worker threads while maintaining a fixed contention level.

At low thread counts, both OCC and 2PL achieve similar throughput because conflicts are limited. As concurrency increases, contention begins to affect system behavior.

In several configurations, Conservative 2PL demonstrates more stable throughput at higher thread counts. This occurs because locking mechanisms serialize conflicting transactions, preventing repeated aborts.

In contrast, OCC may experience throughput degradation under contention because aborted transactions must restart and repeat their execution.



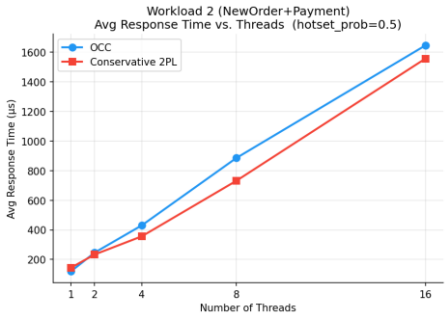


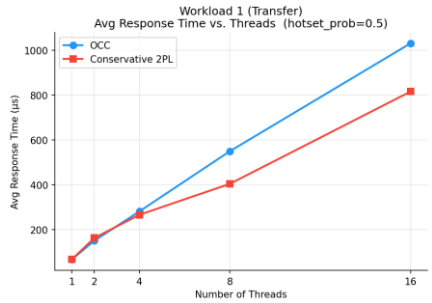
4.3.3 Average Response Time vs Threads

Average response time increases as the number of worker threads grows. Higher concurrency introduces additional synchronization overhead and increases contention for shared records.

OCC generally exhibits slightly higher latency than 2PL in higher-concurrency scenarios. This effect arises because aborted transactions must restart, increasing the total time required to complete a transaction.

2PL avoids transaction restarts but may introduce waiting time due to lock contention. Despite this overhead, the absence of transaction aborts can lead to more predictable latency under higher contention.





4.4 Discussion

The experimental results highlight the fundamental trade-offs between optimistic and locking-based concurrency control mechanisms.

OCC performs efficiently in low-contention environments because transactions can execute without waiting for locks. However, as

contention increases, the cost of transaction restarts becomes significant and negatively impacts both throughput and latency.
Conversely, Conservative 2PL incurs locking overhead but provides stable performance under contention by preventing conflicting transactions from executing concurrently.
"Overall, the results demonstrate that the choice of concurrency control protocol should depend on the expected contention characteristics of the workload. OCC is well suited for workloads with low conflict rates, while 2PL offers more predictable performance when contention is high" [4].

5. Conclusion

This project built and tested a multi-threaded system using two different ways to handle data: Optimistic Concurrency Control (OCC) and Conservative Two-Phase Locking (2PL). Running on top of RocksDB, the system was designed to keep data accurate and organized even when many tasks are running at once. It uses a custom setup that includes a workload runner, specific transaction managers for each style, a "CommitLog" to double-check work for OCC, and a "LockManager" to handle the line-up for 2PL.

To see how they performed, the team ran two different types of workloads under various levels of "traffic." When things were quiet and tasks rarely needed the same data, both protocols performed almost identically. However, as the system got more crowded, their paths diverged. OCC started to struggle with high retry rates because its "check at the end" style led to more failed attempts and slower overall speeds. On the other hand, Conservative 2PL stayed steady by making everyone grab their locks before starting, which prevented those frustrating restarts.

Ultimately, the results confirm a classic trade-off in database design. OCC shines when conflicts are rare because it stays fast and light by skipping the locks. "Conversely, 2PL provides more predictable behavior in workloads with higher contention by preventing conflicting transactions from executing simultaneously. These findings align with established database system design principles and demonstrate the practical implications of concurrency control choices in transactional systems" [4].

6. REFERENCES

[1] Garcia-Molina, H., Ullman, J. D., & Widom, J. (2008). *Database Systems: The Complete Book* (2nd ed.). Pearson Prentice Hall. *Cited by: 251* (via secondary source metadata) http://infolab.stanford.edu/~ullman/dscb.html

[2] Jim Gray and Andreas Reuter. 1992. *Transaction Processing: Concepts and Techniques* (1st ed.). Morgan Kaufmann Publishers Inc., San Francisco, CA, USA. https://dl.acm.org/doi/book/10.5555/573304

[3] Kia Rahmani, Kartick Nagar, Benjamin Delaware, and Suresh Jagannathan. 2021. Repairing Serializability Bugs in Distributed Database Programs via Automated Schema Refactoring. *Proc. ACM Program. Lang.* 5, PLDI, Article 85 (June 2021), 29 pages. https://doi.org/10.1145/3453483.3454028

[4] Kia Rahmani, Kartick Nagar, Benjamin Delaware, and Suresh Jagannathan. 2021. Repairing Serializability Bugs in Distributed Database Programs via Automated Schema Refactoring. *Proc. ACM Program. Lang.* 5, PLDI, Article 85 (June 2021), 29 pages. https://doi.org/10.1145/3453483.3454028

[5] RocksDB Team. 2024. RocksDB Documentation. Retrieved from https://rocksdb.org